UFCD: 0814 **Programação em linguagem SQL avançada**
Formador: João Monge
Aluno: Manoel Silva Barata

1) Exercício 1 — Criar reserva numa transação:

INSERT INTO Reserva(Numero_Reserva,Numero_Cliente,

Dia_Entrada,Dia_Saida)

VALUES (1323480,2004,"2026-03-16 17:00:00","2026-03-18 12:00:00");

INSERT INTO
Reserva_Quartos(Sigla_Hotel,Numero_Quarto,Numero_Reserva,Numero_Pessoas)

VALUES ("IB",2,1323480,2);

INSERT INTO Factura(Numero_Factura,Numero_Reserva,Data_Factura,Valor_Factura)

VALUES (0542380,1323480,"2026-03-16 17:00:00",600);

COMMIT;

```
14
15      VALUES (0542380,1323480,"2026-03-16 17:00:00",600);
16
17 •    COMMIT;
18
19 •    SELECT * FROM Reserva;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

Numero_Reserva	Numero_Cliente	Dia_Entrada	Dia_Saida
1	2001	2026-03-01 17:21:21	2026-03-04 17:21:21
2	2002	2026-03-09 17:21:21	2026-03-11 17:21:21
3	1002	2024-12-01 15:00:00	2024-12-05 11:00:00
4	1004	2026-02-28 17:21:21	2026-03-01 17:21:21
5	2003	2026-03-04 17:21:21	2026-03-06 17:21:21
1323415	2002	2026-03-12 17:00:00	2026-03-13 12:00:00
1323480	2004	2026-03-16 17:00:00	2026-03-18 12:00:00

UFCD: 0814 **Programação em linguagem SQL avançada**
Formador: João Monge
Aluno: Manoel Silva Barata

2) **Testar ROLLBACK:**
   R: Dados não ficam gravados, pois ROLLBACK retorna a operação de inserção.

START TRANSACTION;

INSERT INTO Reserva(Numero_Reserva,Numero_Cliente,

Dia_Entrada,Dia_Saida)

VALUES (1323481,2004,"2026-03-17 17:00:00","2026-03-18 12:00:00");

INSERT INTO
Reserva_Quartos(Sigla_Hotel,Numero_Quarto,Numero_Reserva,Numero_Pessoas)

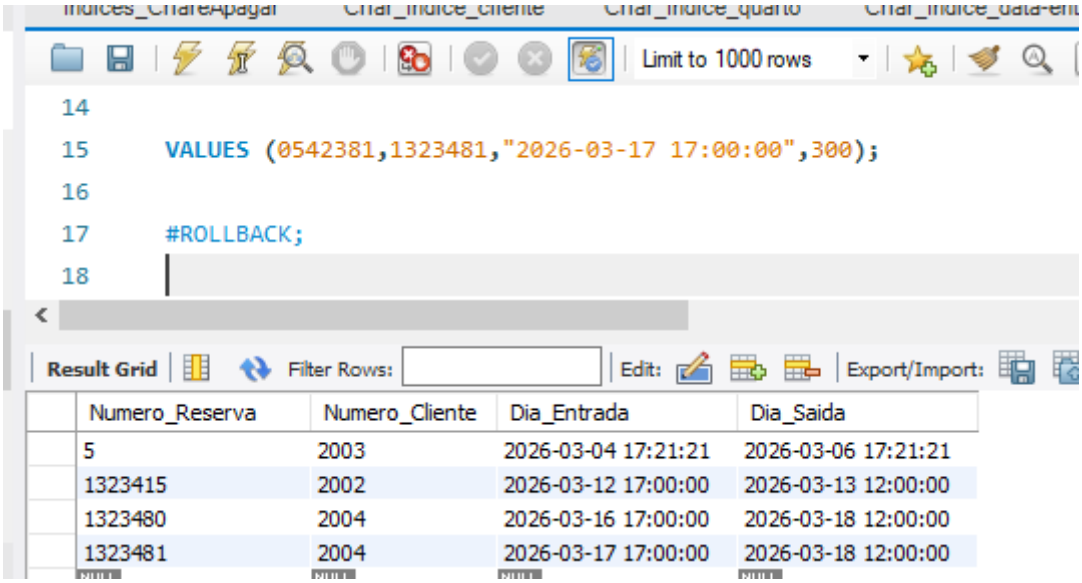
VALUES ("IB",2,1323481,2);

INSERT INTO Factura(Numero_Factura,Numero_Reserva,Data_Factura,Valor_Factura)

VALUES (0542381,1323481,"2026-03-17 17:00:00",300);

ROLLBACK;

SELECT * FROM Reserva;



Numero_Reserva	Numero_Cliente	Dia_Entrada	Dia_Saida
5	2003	2026-03-04 17:21:21	2026-03-06 17:21:21
1323415	2002	2026-03-12 17:00:00	2026-03-13 12:00:00
1323480	2004	2026-03-16 17:00:00	2026-03-18 12:00:00
1323481	2004	2026-03-17 17:00:00	2026-03-18 12:00:00

Indices_CriareApagar | Criar_Indice_cliente | Criar_Indice_quarto | Criar_Indice_data-en

Limit to 1000 rows

```
15    VALUES (0542381,1323481,"2026-03-17 17:00:00",300);
16
17 •  ROLLBACK;
18
19 •  SELECT * FROM Reserva;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

Numero_Reserva	Numero_Cliente	Dia_Entrada	Dia_Saida
4	1004	2026-02-28 17:21:21	2026-03-01 17:21:21
5	2003	2026-03-04 17:21:21	2026-03-06 17:21:21
1323415	2002	2026-03-12 17:00:00	2026-03-13 12:00:00
1323480	2004	2026-03-16 17:00:00	2026-03-18 12:00:00

3) Exercício 3 — Uso de SAVEPOINT

4) Exercício 4 — Numerar quartos por hotel

SELECT Sigla_Hotel, Numero_Quarto, Preco, ROW_NUMBER() OVER (PARTITION BY Sigla_Hotel ORDER BY Numero_Quarto)AS Sequencia FROM Quarto;

Limit to

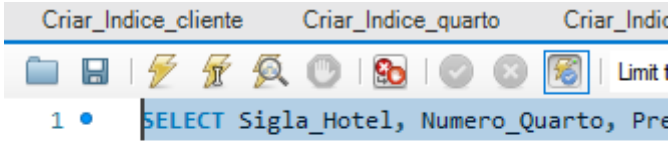
```
1
2 •  SELECT Sigla_Hotel, Numero_Quarto, Prec
```

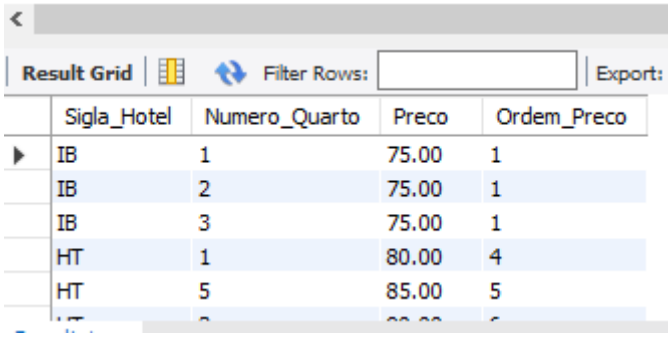
Result Grid | Filter Rows: | Export:

Sigla_Hotel	Numero_Quarto	Preco	Sequencia
HL	4	250.00	4
HL	5	190.00	5
HT	1	80.00	1
HT	2	100.00	2
HT	3	90.00	3

5) Ranking de quartos por preço:

SELECT Sigla_Hotel, Numero_Quarto, Preco, RANK() OVER (ORDER BY Preco ASC)AS Ordem_Preco FROM Quarto

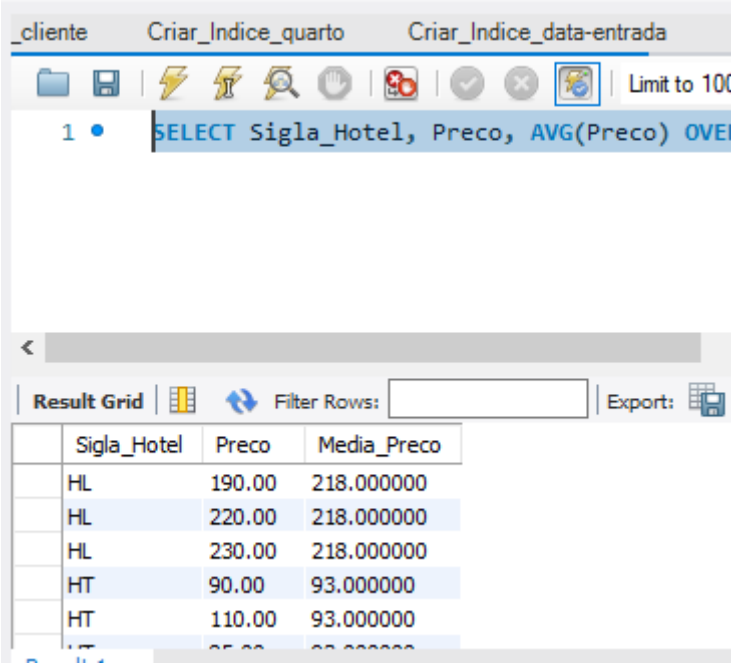




Sigla_Hotel	Numero_Quarto	Preco	Ordem_Preco
IB	1	75.00	1
IB	2	75.00	1
IB	3	75.00	1
HT	1	80.00	4
HT	5	85.00	5

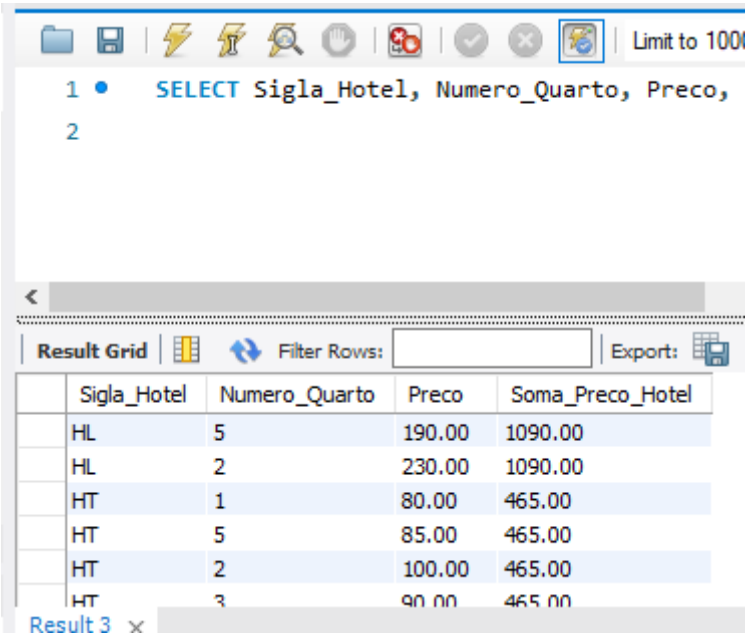
6) Média de preço dos quartos:

SELECT Sigla_Hotel, Preco, AVG(Preco) OVER (PARTITION BY Sigla_Hotel)AS Media_Preco FROM Quarto;



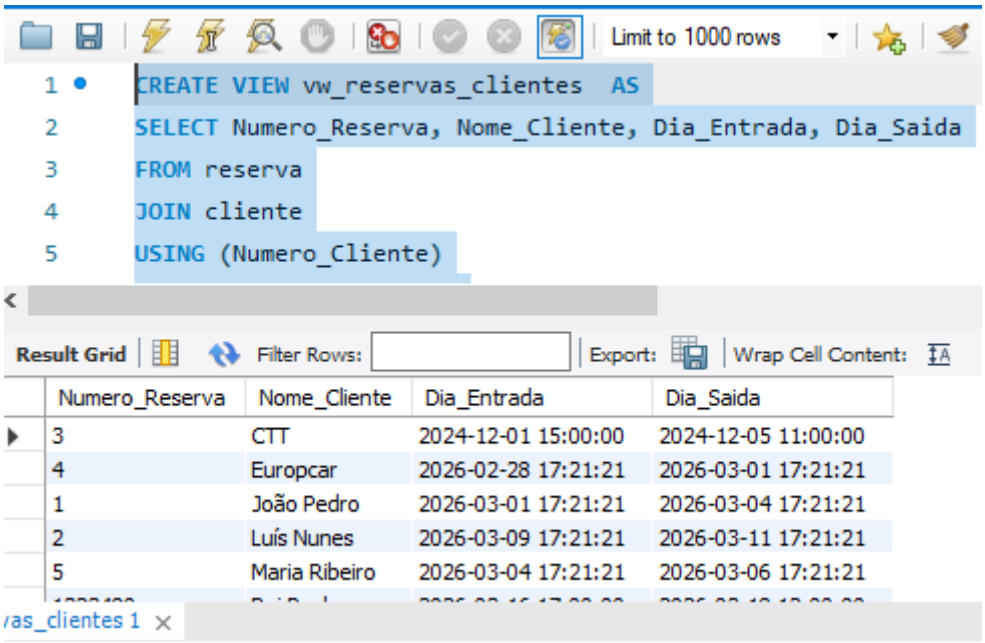
7) Soma acumulada de preços:

SELECT Sigla_Hotel, Numero_Quarto, Preco, SUM(Preco) OVER (PARTITION BY Sigla_Hotel)AS Soma_Preco_Hotel FROM Quarto;



8) View de reservas e clientes:

```
CREATE VIEW vw_reservas_clientes  AS

SELECT Numero_Reserva, Nome_Cliente, Dia_Entrada, Dia_Saida

FROM reserva

JOIN cliente

USING (Numero_Cliente)

GROUP BY Numero_Cliente;
```



9) View de quartos por hotel

```
CREATE VIEW vw_quartos_hotel AS

SELECT Sigla_Hotel, Designacao, Numero_Quarto, Preco

FROM hotel

JOIN quarto

USING (Sigla_Hotel)

GROUP BY Sigla_Hotel;

SELECT * FROM vw_quartos_hotel;
```

```
#CREATE VIEW vw_quartos_hotel AS

SELECT Sigla_Hotel, Designacao, Numero_Quarto, Preco
FROM hotel
JOIN quarto
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conter

Sigla_Hotel	Designacao	Numero_Quarto	Preco
HL	Hilton Lisboa	1	220.00
HT	Hotel Tivoli	1	80.00
IB	Ibis Chiado	1	75.00
ML	Marriott Lisboa	1	180.00
SH	Sheraton Lisboa	1	140.00

10) View de facturas detalhadas

```
CREATE VIEW vw_facturas_detalhadas AS
SELECT Numero_Factura, Nome_Cliente, Numero_Reserva, Valor_Factura, Data_Factura
FROM factura
JOIN reserva
USING (Numero_Reserva)
JOIN cliente
USING (Numero_Cliente)
GROUP BY Numero_Cliente
ORDER BY Numero_Factura ASC;
```

Criar_Indice_data-entrada | Criar_Indice_UNIQUE | Exercicio2 | Exercicio4 | Exercicio5

Limit to 1000 rows

```
USING (Numero_Cliente)
GROUP BY Numero_Cliente
ORDER BY Numero_Factura ASC;

SELECT * FROM vw_facturas_detalhadas;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Numero_Factura	Nome_Cliente	Numero_Reserva	Valor_Factura	Data_Factura
1	João Pedro	1	340.00	2024-11-08 12:00:00
2	Luís Nunes	2	400.00	2024-11-15 09:00:00
3	CTT	3	1550.00	2024-12-05 12:00:00
4	Europcar	4	75.00	2026-02-27 17:21:21
5	Maria Ribeiro	5	200.00	2026-02-27 17:21:21

11) Índice para pesquisa de clientes:

CREATE INDEX idx_nome_cliente

ON Cliente(Nome_Cliente);

Limit to 1000 rows

```
CREATE INDEX idx_nome_cliente
ON Cliente(Nome_Cliente);

SHOW INDEX FROM Cliente;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
cliente	0	PRIMARY	1	Numero_Cliente	A	10	NULL	NULL		BTREE
cliente	1	idx_nome_cliente	1	Nome_Cliente	A	10	NULL	NULL		BTREE

R: Esse índice é útil principalmente em consultas que filtram ou procuram clientes pelo nome. O índice permite que o banco encontre os registros mais rapidamente, sem precisar percorrer toda a tabela, pois organiza os dados em uma estrutura de árvore (geralmente B-Tree), facilitando a localização dos valores.

UFCD: 0814 **Programação em linguagem SQL avançada**
Formador: João Monge
Aluno: Manoel Silva Barata

12) Índice composto para quartos

CREATE INDEX idx_quarto_hotel

ON Quarto(Sigla_Hotel, Numero_Quarto);

```
1 •  CREATE INDEX idx_quarto_hotel
2    ON Quarto(Sigla_Hotel, Numero_Quarto);
3
4 •  SHOW INDEX FROM Quarto;
5
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation
quarto	0	PRIMARY	1	Sigla_Hotel	A
quarto	0	PRIMARY	2	Numero_Quarto	A
quarto	1	idx_quarto_hotel	1	Sigla_Hotel	A
quarto	1	idx_quarto_hotel	2	Numero_Quarto	A

R: Um índice composto pode ser mais eficiente porque organiza os dados considerando as duas colunas juntas, permitindo que o banco localize registros mais rapidamente quando ambas são usadas na consulta. Com esse índice, o banco já possui Sigla_Hotel e Numero_Quarto organizados no mesmo índice, o que permite encontrar o registro diretamente. Se existissem dois índices separados (um para cada coluna), o banco teria que consultar dois índices diferentes e depois combinar os resultados, o que pode ser menos eficiente, especialmente em tabelas grandes.

13) **Índice UNIQUE:**

CREATE UNIQUE INDEX idx_cc
ON Individual(CC);

Criar_Indice_UNIQUE* | Exercicio2 | Exercicio4 | Exercicio5 | Exercicio6 | Exercicio10 | Exercicio7 | Exercicio8

Limit to 1000 rows

```
1 •  CREATE UNIQUE INDEX idx_cc
2    ON Individual(CC);
3
4 •  SHOW INDEX FROM Individual;
5
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_t
individual	0	PRIMARY	1	Numero_Cliente	A	5	NULL	NULL		BTREE
individual	0	CC	1	CC	A	5	NULL	NULL	YES	BTREE
individual	0	idx_cc	1	CC	A	5	NULL	NULL	YES	BTREE

R: Como o índice foi criado com UNIQUE, ele não permite valores duplicados na coluna CC. Se tentar inserir dois registos com o mesmo valor de CC, o primeiro registo será inserido normalmente, mas o segundo será rejeitado pelo banco de dados. Nesse caso, o MySQL mostrará um erro de chave duplicada (Duplicate entry), pois o índice UNIQUE garante que cada valor da coluna seja único na tabela, ajudando a manter a integridade dos dados.

14) Definir nível de isolamento:

15) Comparar níveis de isolamento:

1 - Um dirty read acontece quando uma transação lê dados que foram modificados por outra transação mas ainda não foram confirmados (commit). Se a outra transação fizer rollback, os dados lidos nunca existiram de fato no banco.

2 - Um non-repeatable read ocorre quando uma transação lê o mesmo registro duas vezes e obtém valores diferentes, porque outra transação alterou e confirmou esse registro entre as duas leituras.

3 - Um phantom read acontece quando uma transação executa a mesma consulta duas vezes e, na segunda execução, aparecem novas linhas (registros) que não existiam antes, porque outra transação inseriu dados e fez commit.

4 - O nível de isolamento padrão do MySQL usando o mecanismo InnoDB é REPEATABLE READ.

5 - O nível de isolamento mais forte é SERIALIZABLE, pois ele impede dirty reads, non-repeatable reads e phantom reads, garantindo o maior nível de consistência entre transações.